

Relazione di Laboratorio

Sicurezza delle Applicazioni Web – DVWA (Damn Vulnerable Web Application)

Obiettivo: individuare e sfruttare due vulnerabilità note (Cross-Site Scripting e SQL Injection) sull'applicazione didattica DVWA, impostata a livello di sicurezza "low", per comprenderne il funzionamento e le contromisure.

1. Cross-Site Scripting (XSS) Riflesso

La pagina "XSS reflected" di DVWA richiede all'utente di inserire un nome in un campo di input, il cui valore viene poi restituito e mostrato direttamente nella pagina di risposta, senza alcuna sanificazione (encoding) dell'HTML inserito. Questo comportamento consente di iniettare codice JavaScript arbitrario che verrà eseguito dal browser della vittima.

Payload utilizzato: `<script>alert("I'll show you how deep the rabbit hole goes")</script>`

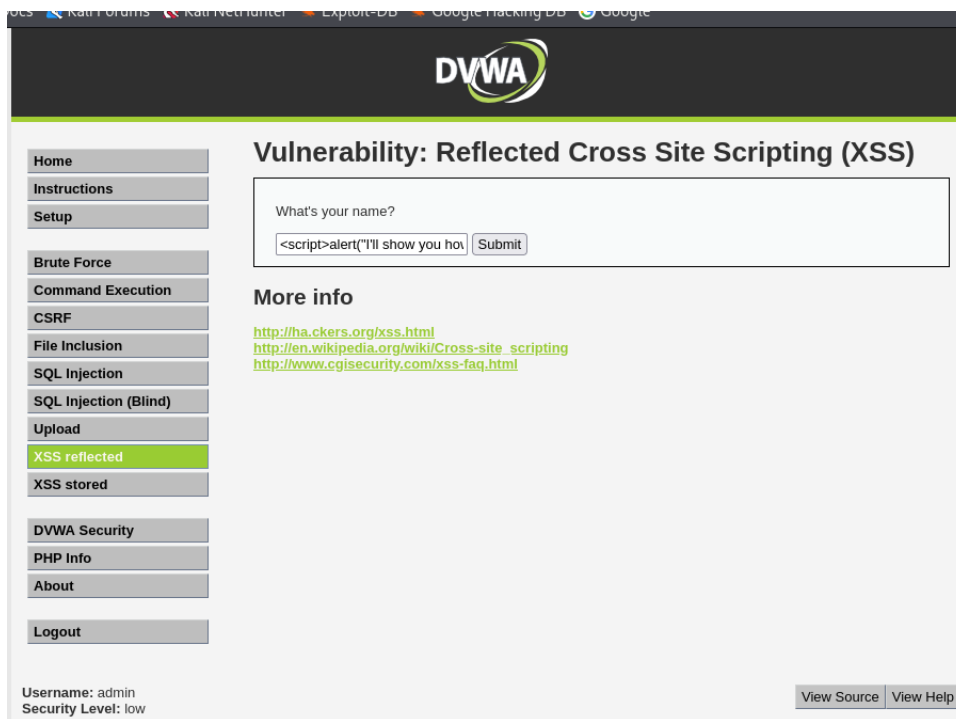


Fig. 1 – Inserimento del payload script nel campo "What's your name?"

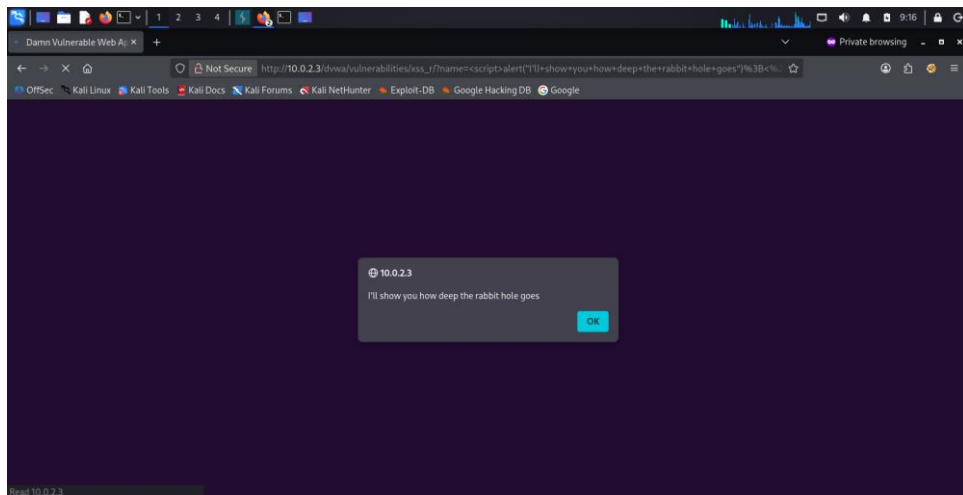


Fig. 2 – Esecuzione del payload: comparsa del popup di alert nel browser

Risultato: il tag `<script>` inserito viene interpretato ed eseguito dal browser, come dimostra il popup di alert mostrato in Fig. 2. Questo conferma che l'input dell'utente non viene filtrato/codificato prima di essere restituito nella pagina, rendendo l'applicazione vulnerabile a XSS riflesso.

2. SQL Injection

La pagina "SQL Injection" di DVWA accetta un User ID e lo inserisce direttamente in una query SQL lato server senza utilizzare parametri preparati (prepared statement) né validazione dell'input. È quindi possibile alterare la logica della query originale inserendo una condizione sempre vera.

Payload utilizzato: `' OR '1'='1' -- a`



Fig. 3 – Inserimento del payload SQL nel campo "User ID"

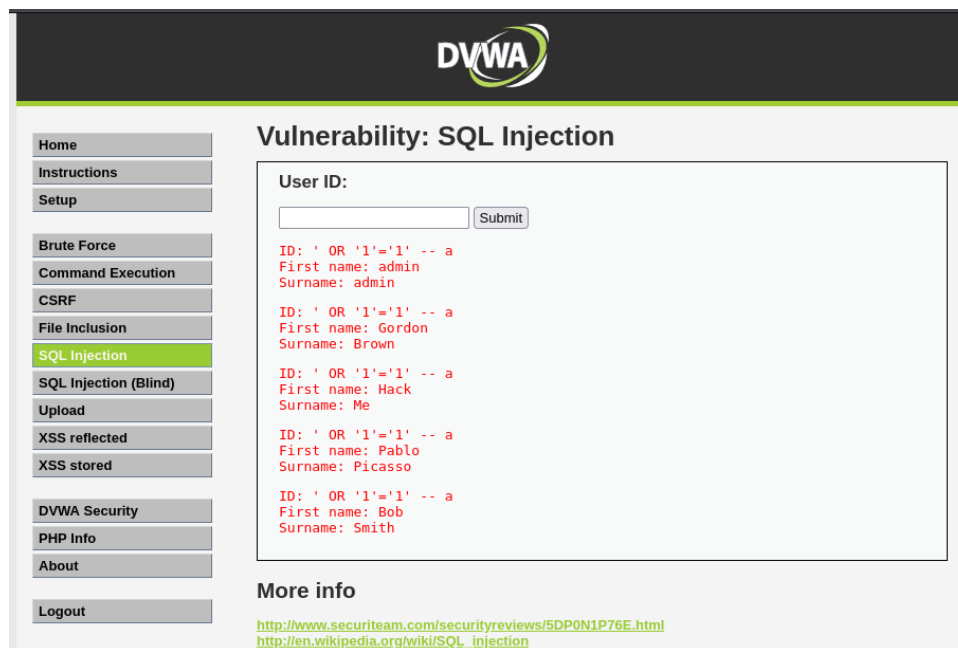


Fig. 4 – Risultato: estrazione di tutti i record della tabella utenti

Risultato: la condizione '1'='1' è sempre vera, quindi la clausola WHERE della query perde la sua funzione di filtro e vengono restituiti tutti i record della tabella (Fig. 4), invece del singolo utente atteso. Ciò dimostra che l'applicazione è vulnerabile a SQL Injection e che un utente malintenzionato potrebbe usare tecniche simili per leggere, modificare o eliminare dati non autorizzati.

3. Conclusioni e contromisure

Entrambe le vulnerabilità derivano dalla mancata validazione/sanificazione dell'input utente lato server. Le principali contromisure sono:

- XSS: codifica (encoding) dell'output HTML, utilizzo di Content Security Policy (CSP) e validazione dell'input.
- SQL Injection: utilizzo di query parametrizzate (prepared statement) e principio del minimo privilegio per l'utente del database.